Лабораторна робота №2 — Повний гайд до захисту

Предметна область: SportsBooking (Бронювання спортивних майданчиків)
Оцінка: 10 / 10 обов'язкових балів + +3 бали за Docker Compose = **13 балів**

Суть роботи: Ми винесли операційний модуль бронювань із моноліту в окремий незалежний мікросервіс `BookingService` (порт `5003`, окрема БД `miniproject_booking`), прибрали зовнішні ключі між базами, додали перевірку каталогу з таймаутом 2 с, захистили створення ідемпотентністю (`Idempotency-Key`), зберегли знімок цін (`ItemSnapshot`), перемкнули шлюз `ApiGateway` (:5000) і довели 10 інтеграційними тестами (`dotnet test`), що система працює без регресій.

Розділ 1. Карта коду — куди дивитися і що показувати

Функціонал	Файл у проєкті	Що саме показати викладачу
Контролер	`src/BookingService/Controllers/BookingsController.cs`	Маршрути `/api/bookings`, атрибут `[Authorize]`, `UserId` витягується з токена (не з тіла).
Бізнес-логіка	`src/BookingService/Services/BookingServiceImpl.cs`	Метод `ValidateBusinessRules` (дата в майбутньому, години 1..24), перевірка перетину слотів, ідемпотентність.
Клієнт каталогу (таймаут 2 с)	`src/BookingService/Clients/CatalogClient.cs`	Дедлайн `timeoutCts.CancelAfter(TimeSpan.FromSeconds(2))`, 3 випадки помилок: 404, таймаут, недоступність.
База даних та індекси	`src/BookingService/Data/BookingDbContext.cs`	`HasPrecision(10,2)`, статус рядком `HasConversion<string>()`, унікальний індекс на `IdempotencyKey`, відсутність FK.
Міграція та автоінкремент	`src/BookingService/Migrations/20261019174922_InitialCreate.cs`	Рядок 84: `migrationBuilder.Sql("ALTER TABLE Bookings AUTO_INCREMENT = 100;");` та перенесення старих записів 1, 3, 4.
Обробка помилок	`src/BookingService/Infrastructure/GlobalExceptionHandler.cs`	Реалізація `IExceptionHandler`: клієнт бачить `ProblemDetails` без витоку stack trace.
Шлюз Ocelot	`src/ApiGateway/ocelot.json`	Маршрут `/api/bookings/{everything}` перемкнено на порт `5003` замість 5100.
10 тестів	`tests/BookingService.IntegrationTests/BookingIntegrationTests.cs`	Всі 10 тестів `Scenario01_...` до `Scenario10_...`, перевірка статус-коду та тіла JSON.
Стаб каталогу	`tests/BookingService.IntegrationTests/ApiFactory.cs`	`FakeCatalogClient`, тестова база `miniproject_booking_test`, генерація тестових JWT токенів.
Baseline знімок	`tests/baseline/baseline.http`	Запити до моноліту та збережені файли відповідей у `tests/baseline/`.
ADR-0003	`docs/adr/0003-operations-extraction.md`	Таблиця «наші дані / чужі дані / технічна дрібниця» та обґрунтування винесення.
Порівняння	`docs/baseline-diff.md`	Таблиця порівняння «Було / Стало» для всіх 10 сценаріїв — доказ відсутності регресій.
Docker Compose (+3 бали)	`docker-compose.yml`	Підняття однією командою MySQL (порт 3307), Auth (5001), Catalog (5002), Booking (5003), Gateway (5000).

Розділ 2. Що конкретно вводити в Swagger (Покроковий приклад)

Відкрийте в браузері: **http://localhost:5003/swagger**

Крок А. Авторизація в Swagger (отримати і вставити токен)

Оскільки всі операції захищені атрибутом `[Authorize]`, спочатку треба авторизуватися:

1. Отримайте токен адміністратора, виконавши команду в терміналі (або у Swagger AuthService `:5001`):
   ```
   Invoke-RestMethod http://localhost:5001/api/v1/auth/login -Method POST -ContentType 'application/json' -Body '{"email":"admin@sportsbooking.com","password":"Admin123!"}'
   ```
   Скопіюйте довгий рядок `accessToken` із відповіді.
2. У Swagger BookingService (`:5003/swagger`) вгорі праворуч натисніть зелену кнопку **Authorize** 🔓.
3. У полі *Value* вставте ваш токен:
   ```
   Bearer ВСТАВИТИ_ВАШ_ТОКЕН_СЮДИ
   ```
   (або просто сам токен без слова Bearer) і натисніть кнопку **Authorize**, а потім **Close**. Замок закриється 🔒.

Крок Б. Створення нового бронювання (POST /api/bookings)

1. Розкрийте зелений блок **POST /api/bookings**.
2. Натисніть кнопку **Try it out** праворуч.
3. У полі *Request body* вставте такий готовий JSON:
   ```
   {
     "venueId": 1,
     "startTime": "2027-05-10T14:00:00Z",
     "hours": 2
   }
   ```
4. (Опціонально) У полі заголовка `Idempotency-Key` можна написати `demo-key-1`.
5. Натисніть велику синю кнопку **Execute**.
6. **Результат:** статус `201 Created`, у відповіді повернеться бронювання з `id` (наприклад, 100), збереженим знімком майданчика `"venueName": "Central Football Field"` та порахованою сумою `"totalPrice": 1000.00`.

Крок В. Перегляд списку своїх бронювань (GET /api/bookings)

1. Розкрийте синій блок **GET /api/bookings**.
2. Натисніть **Try it out**.
3. Параметри можна залишити за замовчуванням (`page: 1`, `pageSize: 20`, або ввести `status: Confirmed`).
4. Натисніть **Execute**.
5. **Результат:** статус `200 OK` і JSON зі списком ваших бронювань та пагінацією.

Крок Г. Скасування бронювання (POST /api/bookings/{id}/cancel)

1. Розкрийте блок **POST /api/bookings/{id}/cancel**.
2. Натисніть **Try it out**.
3. Введіть `id` вашого щойно створеного бронювання (наприклад, `100`).
4. Натисніть **Execute**.
5. **Результат:** статус `200 OK`, статус бронювання змінився на `"status": "Cancelled"`.
6. Якщо натиснути **Execute** ще раз — повернеться помилка `400 Bad Request` («Бронювання вже скасовано»).

Розділ 3. Покроковий план захисту (7 дій перед викладачем)

Дія 1. Показати tests/baseline/ та історію комітів у Git

- Покажіть файл `tests/baseline/baseline.http` та збережені JSON-відповіді моноліту.
- У GitHub Desktop покажіть вкладку **History** — перший коміт зафіксував baseline до коду сервісу.
- **Фраза для викладача:** *«Спочатку ми зафіксували базову поведінку (baseline) моноліту, щоб мати еталон для порівняння, і лише після цього виносили код нового сервісу».*

Дія 2. Підняти систему і показати власну БД з перенесеними даними

- Покажіть запущений Docker Desktop (група `sportsbooking` горить зеленим) або виконайте:

```
docker compose up -d
```

- Покажіть таблицю в MySQL (через Workbench або команду в консолі):

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p292929 -e "SELECT Id, VenueName, PricePerHour, TotalPrice, Status FROM miniproject_booking.Bookings;"
```

- **Фраза для викладача:** *«BookingService має власну базу miniproject_booking. Старі записи з моноліту (Id 1, 3, 4) перенесені міграцією, а AUTO_INCREMENT піднято до 100, тому нові записи не перезапишуть старі».*

Дія 3. Запустити тести (dotnet test) і показати «злам логіки»

- Виконайте в терміналі:

```
dotnet test
```

  Результат: `Passed! - Failed: 0, Passed: 10, Skipped: 0` (10 із 10 тестів зелені).
- **Якщо викладач скаже: «Зламай рядок і покажи, що тест червоніє»:**
  1. У файлі `src/BookingService/Services/BookingServiceImpl.cs` (рядок 268) тимчасово закоментуйте рядок:
     `// if (startUtc <= DateTime.UtcNow)`
  2. Запустіть `dotnet test` → тест `Scenario04` впаде червоним.
  3. Розкоментуйте рядок назад → тести знову зелені. Це залізобетонний доказ якості тестів.

Дія 4. Виконати запити через шлюз (порт 5000)

- У файлі `tests/baseline/baseline.http` натискайте кнопки **Send Request** навпроти запитів. Запити проходять через `http://localhost:5000`.
- Покажіть файл `docs/baseline-diff.md` — відповіді шлюзу повністю збігаються з монолітом.

Дія 5. Зупинити CatalogService і показати відмовостійкість (503 помилка)

- У Docker Desktop натисніть кнопку **Stop** (квадратик) на контейнері `sportsbooking-catalog`.
- У Swagger або через `baseline.http` надішліть створення бронювання.
- **Результат:** через рівно 2 секунди (наш дедлайн) сервіс не падає, а повертає свідомий:
  `503 Service Unavailable: "Сервіс каталогу недоступний"`.
- **Фраза для викладача:** *«Завдяки таймауту у 2 секунди в CatalogClient потік не зависає назавжди, система не падає, а повертає коректний 503 код».*
- Запустіть контейнер `sportsbooking-catalog` назад.

Дія 6. Показати Swagger UI

- Відкрийте `http://localhost:5003/swagger` — покажіть описи, коди відповідей 200/201/400/401/404/503 та кнопку Authorize.

Розділ 4. Готові відповіді на ВСІ 11 контрольних питань (Розділ 14)

1. Що таке шов (seam) і за якими ознаками ви його шукали у своєму моноліті?

Відповідь: Шов — це межа в моноліті, де його можна розрізати без зміни коду інших частин. Ми поділили сутності моноліту на три групи: *наші дані* (Bookings — перенесли у власну БД), *чужі дані* (Venues, Users — читаємо по мережі або з JWT) та *спільні дрібниці* (статуси — здублювали локально в сервісі).

2. Чому між базами різних сервісів не роблять зовнішніх ключів? Що використовують замість них?

Відповідь: Мікросервіси мають бути автономними (принцип Database per Service), їхні БД можуть бути на різних серверах. Зовнішній ключ (FK) на рівні СУБД створив би жорстку залежність. Замість нього зберігають числове поле `VenueId`, перевіряють його по мережі через `CatalogClient` і фіксують знімок назви та ціни (ItemSnapshot).

3. Що таке характеризаційний тест і чим він відрізняється від звичайного?

Відповідь: Звичайний тест перевіряє код за вимогами («як має бути»). Характеризаційний тест (baseline) знімає фактичну поведінку існуючої системи («як вона відповідає сьогодні»), щоб після винесення переконатися, що жодна деталь і жоден код помилки не змінилися.

4. Чим інтеграційний тест відрізняється від модульного і від e2e-тесту?

Відповідь: Unit-тест перевіряє один клас в ізоляції без бази. Інтеграційний тест перевіряє конвеєр ASP.NET Core + валідацію + EF Core + тестову базу в пам'яті тестового процесу. E2E-тест перевіряє всю систему наскрізно через шлюз з реальними процесами.

5. Для чого потрібен WebApplicationFactory і що саме ви в ньому підмінили?

Відповідь: Він піднімає веб-застосунок у тестовому процесі без відкриття сокетів і без `dotnet run`. Ми підмінили в ньому: 1) робочу БД на окрему тестову `miniproject_booking_test`; 2) клієнта каталогу на стаб `FakeCatalogClient` із готовими відповідями.

6. Що таке ідемпотентність? Наведіть ідемпотентний і неідемпотентний метод.

Відповідь: Це властивість операції давати той самий результат при повторних викликах. Ідемпотентні за специфікацією: `GET`, `PUT`, `DELETE`. Неідемпотентний: `POST`. Щоб зробити наш POST ідемпотентним, ми додали заголовок `Idempotency-Key` та унікальний індекс у БД.

7. Навіщо виклику до сусіднього сервісу потрібен дедлайн? Що станеться без нього, якщо сусід «завис»?

Відповідь: Дедлайн (таймаут 2 с) обмежує час очікування. Без нього потік сервера чекатиме вічно, закінчаться вільні потоки пулу і наш сервіс теж впаде (каскадна відмова). Завдяки дедлайну ми повертаємо клієнту контрольований 503 код.

8. Чому в операції зберігають копію назви та ціни об'єкта каталогу, а не лише його Id?

Відповідь: Це патерн **ItemSnapshot**. Якщо через півроку в каталозі піднімуть ціну майданчика, старі бронювання клієнтів не повинні перераховуватися за новими тарифами. Знімок гарантує незмінність фінансової історії.

9. Як ви перенесли старі дані й чому підняли автоінкремент?

Відповідь: Записи зі старої бази (Id 1, 3, 4) внесені в міграцію `InitialCreate` через `HasData`. Командою `ALTER TABLE Bookings AUTO_INCREMENT = 100` ми підняли стартовий номер, щоб нові Id починалися зі 100 і не перетиналися зі старими.

10. Що таке Strangler Fig і на якому його кроці ви зараз перебуваєте?

Відповідь: Це патерн поступової заміни моноліту мікросервісами через API Gateway. Ми перебуваємо на кроці, коли сервіси (Auth, Catalog, Booking) винесені, шлюз перемкнено на них, а старий моноліт залишено поруч як страховку на час перевірки.

11. Як ви дізналися б, що винесення зламало поведінку, якби не було baseline-файлів?

Відповідь: Без baseline це помітили б лише реальні клієнти після релізу (зміна регістру JSON, зниклий 404 код, зламана валідація). Базовий знімок дозволяє автоматично виявити регресії ще під час розробки.